ACDC IPEX Negotiation

There are several mechanisms that could be used to negotiate a graduate disclosure. These include: attribute path lists, aggregate block label lists, metadata ACDCs, Schema, and Decompositions of Schema.

The problem with negotiating with schema docompositions is that attributes appear disjoint in a schema so pathing to indicate an attribute gets complicated quickly and the pathing logic has to understand JSON schema syntax at a deep level. Whereas in a metadata ACDC the attribute paths are not disjoint so indicating an attribute at any nesting level can be handled with a path string.

So it feels like to me that negotiation should be using Composed Schema as type. So only the SAID of the schema is provided and assume the full schema is cached or attached. Then use pathing of attributes to request the level of partial disclsure and metadata ACDCs for terms negotiation prior to disclosure.

Desired attributes are requested as a list of paths of each attribute or block. If the path is a block then the whole block is meant to be disclosed. If the path is a field in a block only the fields in that block at that field level needs to be disclosed. Any fields whose value is a nested block in that block does not need to be expanded and disclosed.

With nested partial disclosure in an ATT section in order to disclose any field in a block all the simple fields in that block must also be disclosed but non-simple fields i.e. fields whose value is a nested block need only disclose the SAID of the block. So specifiying any simple field via a path would get you all the simple fields in the block holding the specified leaf of the path.

Whereas specifying a block (directory) would be short hand for disclosing the full tree attached all the way down.

For ATT section ACDCs the pathing gets simple since its an array and only the unique field label needs to be specified to designate a given block.

For example

```
'a':
{
    'd': "Ebslde",
    'u': "Abldle"
```

```
     'i': "EDdecd",
     "grades":
     {
          "d": "Efghi",
          "u":  "Abdefs",
          "gpa":  3.7,
         "transcript":
         {
              "d": "EBDE",
              "u":  "ACEF",
                "math": 3.5,
               "english":3.8",
         }
      }
 }
```

}

SAID of composed schema as type defines the ACDC in the apple: EABCDSS...
Attribute path list defines what attributes are being asked for: ['a/i', 'a/grades/']
So apply is asking for the top level of the "a" section via 'a/i' or '/a/i' and for the full grades block and any nested subblocks of grades via 'a/grades/' or '/a/grades'
Or because grades is nested down from the top level of 'a'. A path list of simply ["/a/grades/"

Where as a path list of merely ['/a/grades/'] subsumes the other path list.

To get gpa (and the issuee aid) without the grades detail would be simply ['/a/grades/gpa']

In response to an offer, the chain to the apply could either reply with the same schema and path list, or the rule could be that an offer chained to the same apply that leaves the path list None implies it is offering the same path list.
and then the offer includes a metadata acdc that exposes the rule section and the att section said but not the pathed attributes.
The agree then chains to the offer which indicates agreement
The grant then provides an uncompacted version of the ACDC with the appropriate level of disclosure.

To generalize this to apply to those who want graduated disclosure of multiple ACDCs in a chain, I suggest a dict
where the item label is the SAID of the ACDC in the chain, and the value of the item is the path list of disclosed attributes.

The path list is not limited to attribute and aggregate sections; paths could include edge and rule

section paths.

For example, two chained acdcs one with an attribute section and one with an aggregate section

{
"EABCDED": ["/a/gpa"],
"EBQRSTR" : ["/A/i","/A/photo"]
}

The path dict requests the gpa field in the ACDC with schema "EABCDED" which means that anything on the path to the "gpa" field must also be disclosed and also requests the selectively disclosable Issuee and photo blocks of the schema "EBQRSTR"

We cannot use ACDC SAIDs in an application in general because the actual ACDC SAID is assumed not to be known by the applicant. But for defined ACDCs, as would be the case for any ACDCs governed by an EGF, the SAIDS of the ACDC schemas would be known. And so the request is for an unidentified ACDC but with a given schema SAID and the paths indicated by the path list.

When the ACDC disclosure includes a bespoke ACDC the ap
When the ACDC disclosure includes a bespoke ACDC, the applicant won't necessarily know the SAID of its schema in the application, but the initial offer can provide it via a bespoke metadata ACDC, and subsequent apply messages can negotiate on that bespoke ACDC schema SAID.

The paths shown use the "/' delimeter but for CESR compatibility we could use the CESR Pathing BASE64 compatible path delimeter "-". Or if we do a little work to name the path list field normatively we could have the tooling auto convert from "/" delimited paths to more compact "-" delimited paths when using CESR serialization of the exchange message.
Or we allow either delimeter and just inject and extract the path as a tuple for reasoning so we are delimiter independent at the reasoning level..

Clarifications

Issuee AID for aggregative (selectively disclosable) ACDCs

I believe for the core SEDI identity credential, the appropriate choice is to use the selectively disclosable Aggregate section. The state of Utah has not decided, but my suggestion is that it be selectively disclosable, given that most use cases seem to fit that model. In many cases, the only disclosed field value would be the issuee AID field.

The "i" field for Issuee is optional. When not present, it means the ACDC is issued to whom it may concern (untargeted).

The operator I2I is intended to work for either att or agg ACDCs when an "i" field is present.

In the case where no "i" issuee aid field is provided, the I2I operator on an edge would fail.

This is a case where the term "holder" is confusing, because anyone could "hold" and present an untargeted ACDC. Unlike W3C credentials, where the "holder" binding is variable, with both weak and strong forms. An ACDC either has a cryptographically verifiable binding (i.e. strongest) via an "i" field with the issuee aid or there is no cryptographically verifiable binding. It's a boolean concept.

To elaborate:
An attributive ACDC has an "a" section, and if its "a" section has an "i" field at its top level, then the value of that "i" field is the issuee AID.
An aggregative ACDC has an "A" section, and if the second block in its "A" section has an "i" field, then the value of the "i" field is the issuee AID.
An ACDC can only be either attributive or aggregative, not both (mutually exclusive)

The spec should be clarified to precisely describe where the 'i' field in an Agg section is found when present.
The issuee in an ACDC with Agg section (selectively disclosable) is the "i" field in the second (index 1) attribute block.

The SerderACDC class has two relevant properties that provide the issuer AID as appropriate for the type of ACDC (attributive or aggregative). The properties are the .iseaid and .issuee. The .iseaid property returns the issuee aid or None (when the AID is untargeted). The .issuee property returns a Prefixer Matter primitive instance whose .qb64 property is the issuee aid or None. For example, see these asserts in the test_selective_disclosure_aggregate_JSON()

```
    assert acdc.iseaid == BOB
    assert acdc.issuee.qb64 == BOB
```

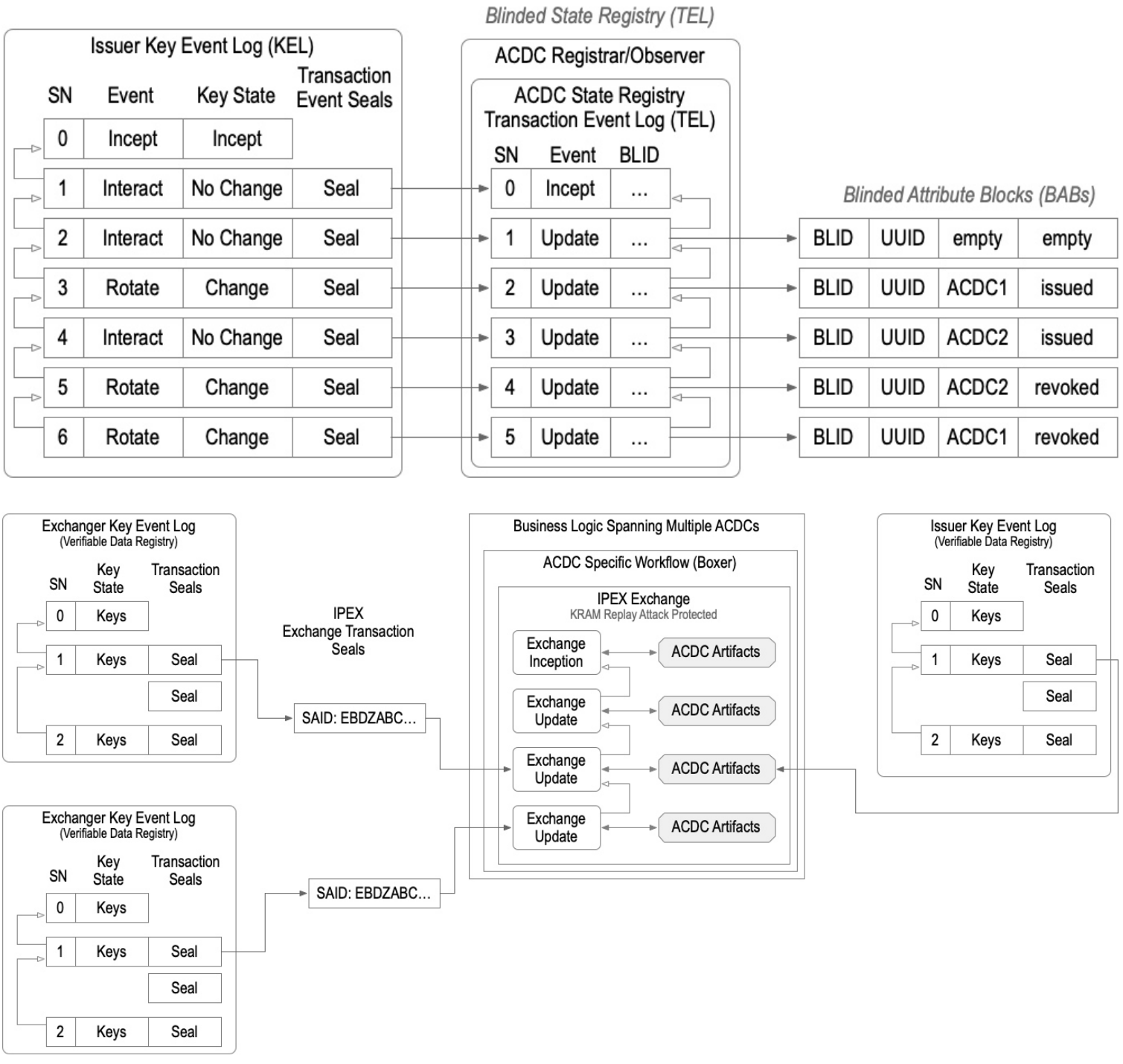
Presentation Registries

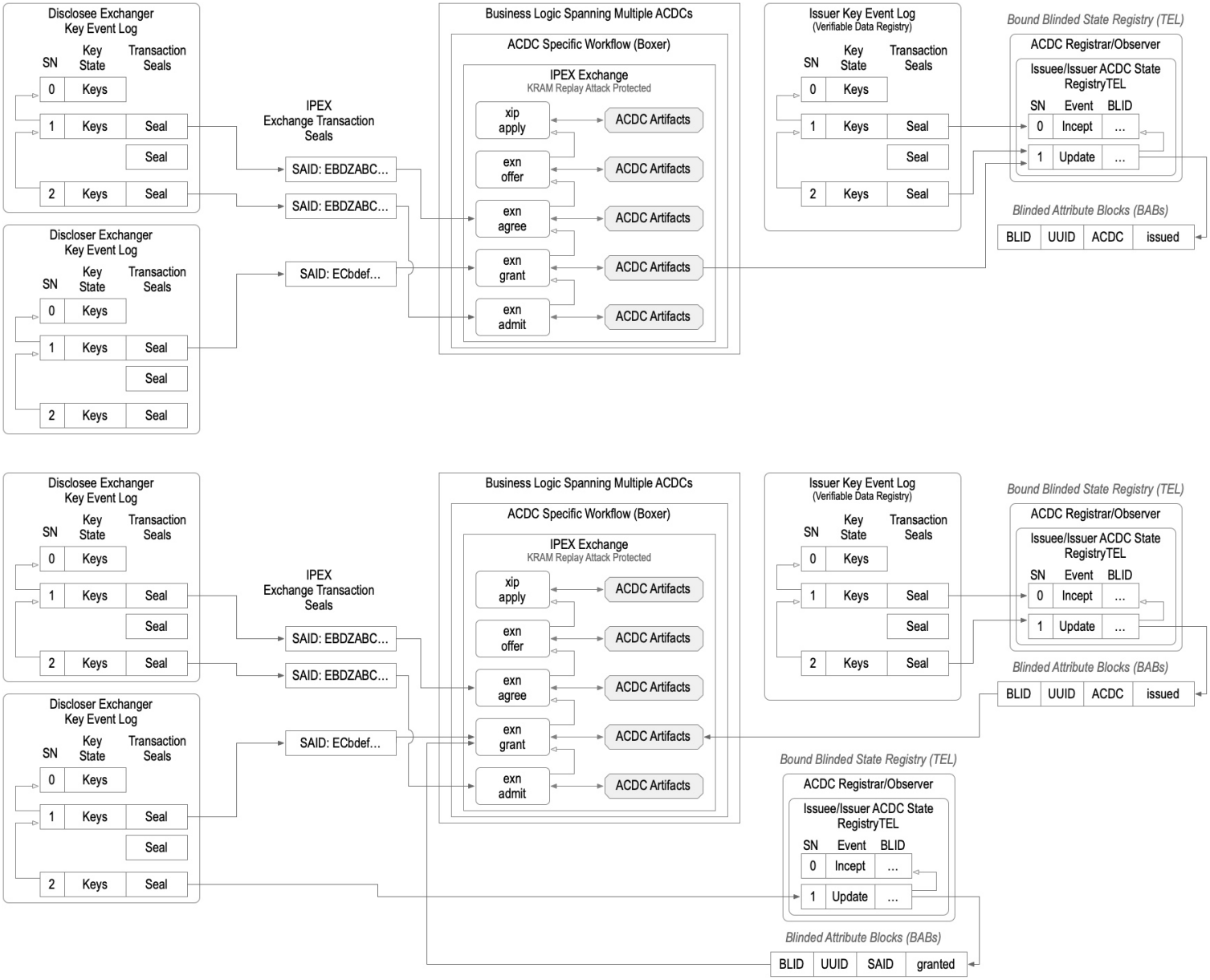
However, there is some discussion about how to manage presentation registries. Should they be optional on a citizen by citizen basis. Or even a bulk issuance set member basis. A presentation registry enables the Issuee to detect impersonation fraud of a compromise of its signing keys, so

high stakes presentations want to to use a presentation registry, which requires that the "rd" field be populated at the time of issuance to force the verifier to verify the presentation against the presentation registry controlled by the issuee.

This leads me to believe that we need to find a high-stakes use case where the Issuee would want to enforce a presentation registry. Maybe a drug prescription presentation where the prescription is either an ACDC or the cargo of a bespoke ACDC.

Diagrams




Disclosee Exchanger
Key Event Log
SN
Key State
Transaction Seals
0
Keys
1
Keys
Seal
Seal
2
Keys
Seal
Discloser Exchanger
Key Event Log
IPEX
Exchange Transaction Seals
SAID: EBDZABC…
SAID: EBDZABC…
SAID: ECbdef…
Business Logic Spanning Multiple ACDCs
ACDC Specific Workflow (Boxer)
IPEX Exchange
KRAM Replay Attack Protected
xip apply
exn offer
exn agree
exn grant
exn admit
ACDC Artifacts
Issuer Key Event Log
(Verifiable Data Registry)
Bound Blinded State Registry (TEL)
ACDC Registrar/Observer
Issuee/Issuer ACDC State RegistryTEL
Event
BLID
Incept
Update
…
Blinded Attribute Blocks (BABs)
BLID
UUID
ACDC
issued

Disclosee Exchanger
Key Event Log
SN
Key State
Transaction Seals
0
Keys
1
Keys
Seal
Seal
2
Keys
Seal
Discloser Exchanger
Key Event Log
IPEX
Exchange Transaction Seals
SAID: EBDZABC…
SAID: EBDZABC…
SAID: ECbdef…
Business Logic Spanning Multiple ACDCs
ACDC Specific Workflow (Boxer)
IPEX Exchange
KRAM Replay Attack Protected
xip apply
exn offer
exn agree
exn grant
exn admit
ACDC Artifacts
Issuer Key Event Log
(Verifiable Data Registry)
Bound Blinded State Registry (TEL)
ACDC Registrar/Observer
Issuee/Issuer ACDC State RegistryTEL
Event
BLID
Incept
Update
…
Blinded Attribute Blocks (BABs)
BLID
UUID
ACDC
issued
Bound Blinded State Registry (TEL)
ACDC Registrar/Observer
Issuee/Issuer ACDC State RegistryTEL
Event
BLID
Incept
Update
…
Blinded Attribute Blocks (BABs)
BLID
UUID
SAID
granted